UCD Michael Smurfit Graduate Business School

Name: Shruti Avinash Punjabi
Student ID : 25201664
MSc in Financial Data Science

Module: Data Science for Trading & Risk Management
FIN42110: Homework 1
Module Leader: Richard McGee
Due: February 13th 2023

Table of Content

1. Introduction 3
2. Data Analysis 3
3. Anomaly Check 6
4. Daily Trading volume aggregated with exchange. 6
5. Realized Variance Construction 7
6. Volatility Forecasting 8
7. Visualisations 8
8. Appendix 13

1. Introduction

This report analyses high frequency bitcoin trading data from multiple cryptocurrency exchanges. The objectives of the project is to visualise price and volume dynamics, construct realized variance from intraday prices and examine the predictability of future volatility using simple forecasting models.

2. Data Analysis

High Frequency Bitcoin Price data from multiple exchanges (Coinbase, BitFinex, BitStamp, HitBTC and Kraken) were loaded and visualised. Exchanged identifiers were mapped to consistent names and timestamps were converted to datetime format to facilitate time-based aggregation.

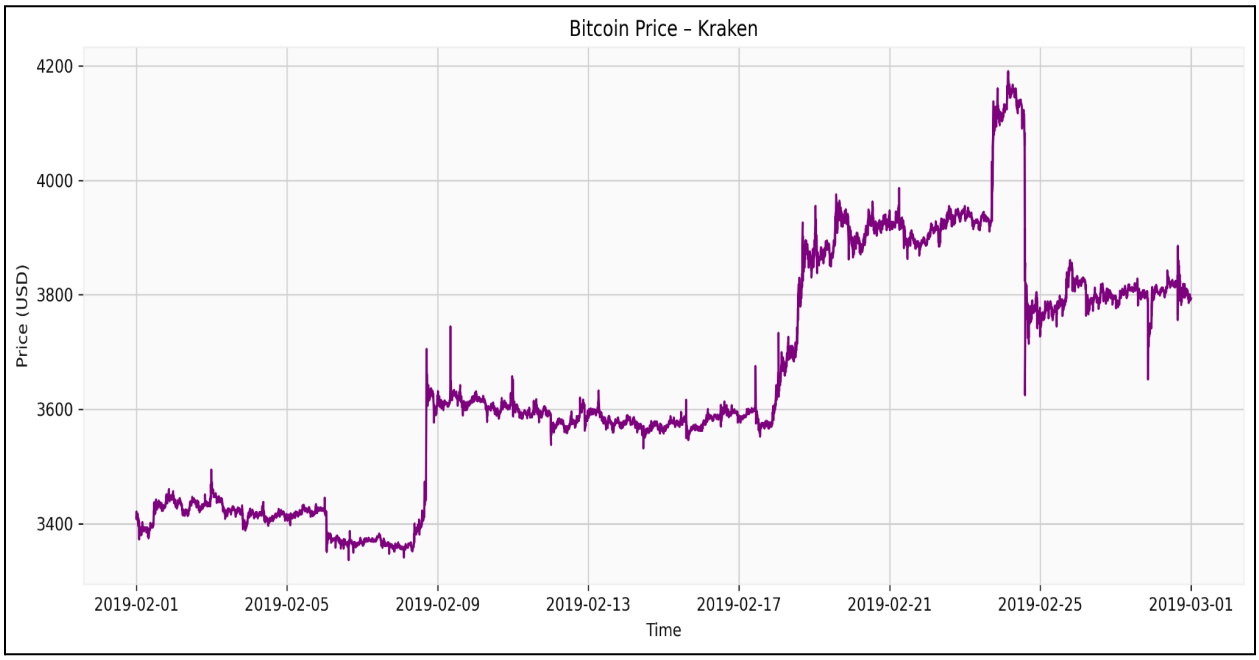

Figure 2.1 Bitcoin price time series for Kraken

Figure 2.2 Bitcoin price time series for HitBTC

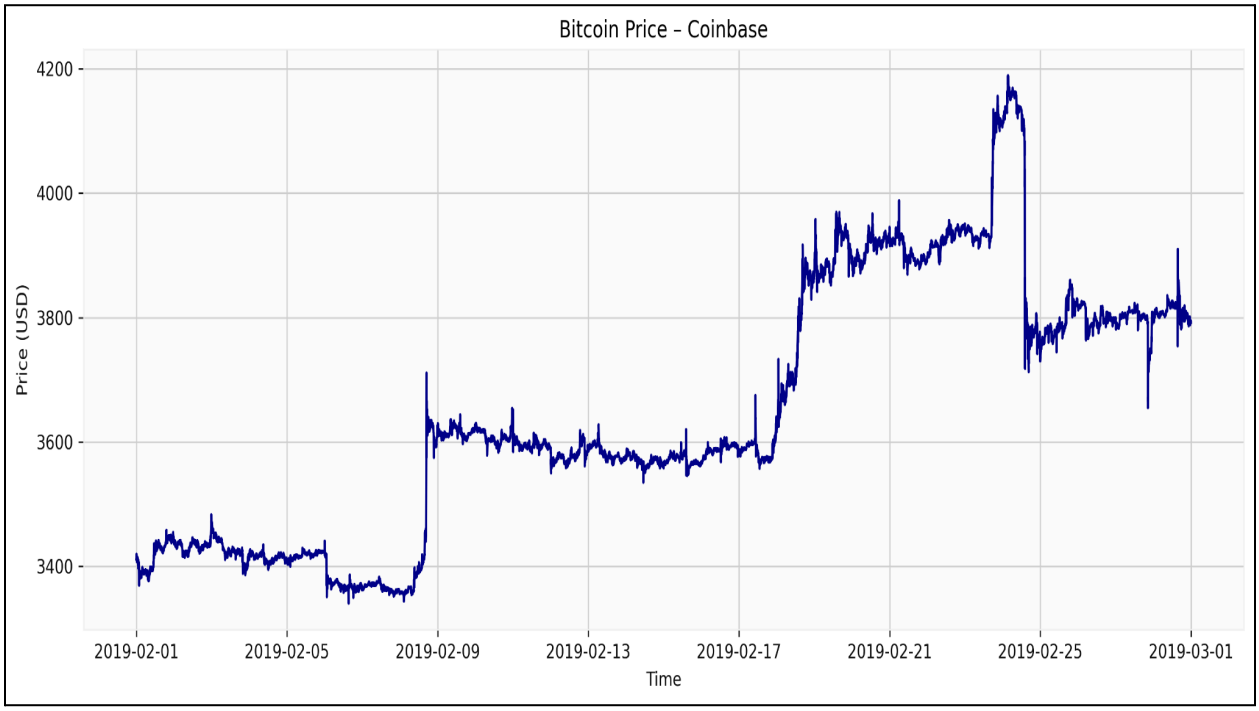

Figure 2.3 Bitcoin price time series for Coinbase

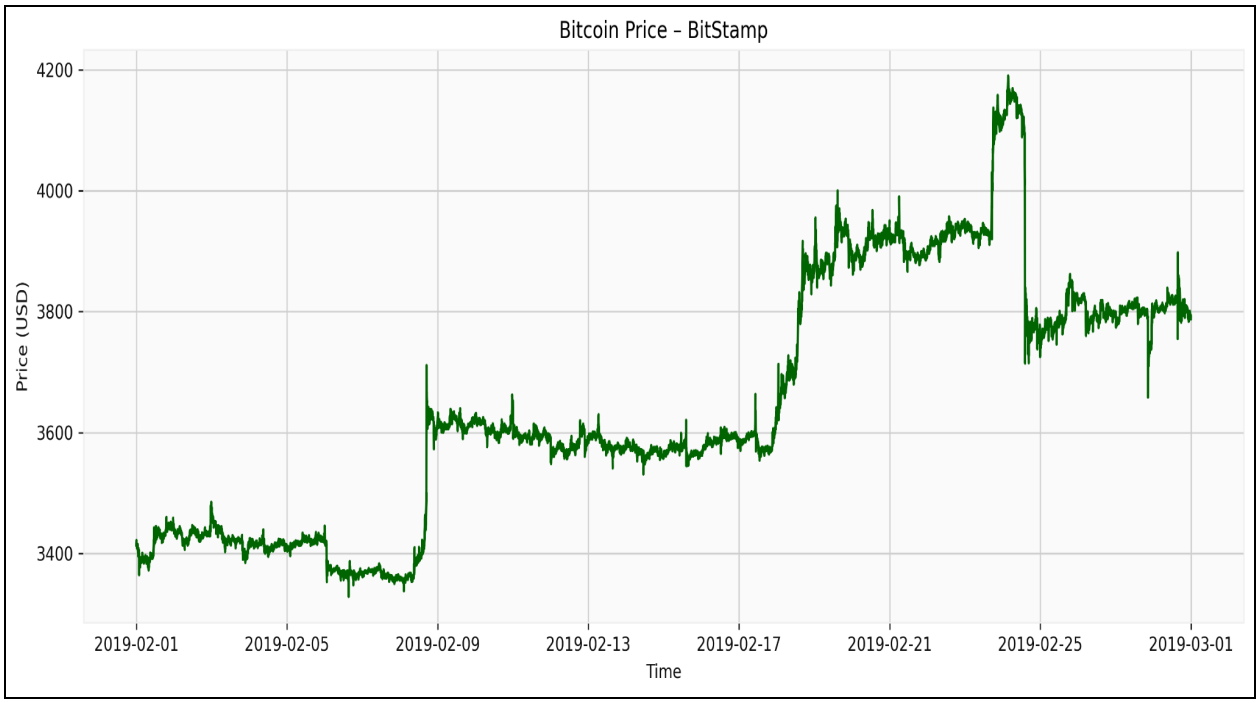

Figure 2.4 Bitcoin price time series for Bitstamp

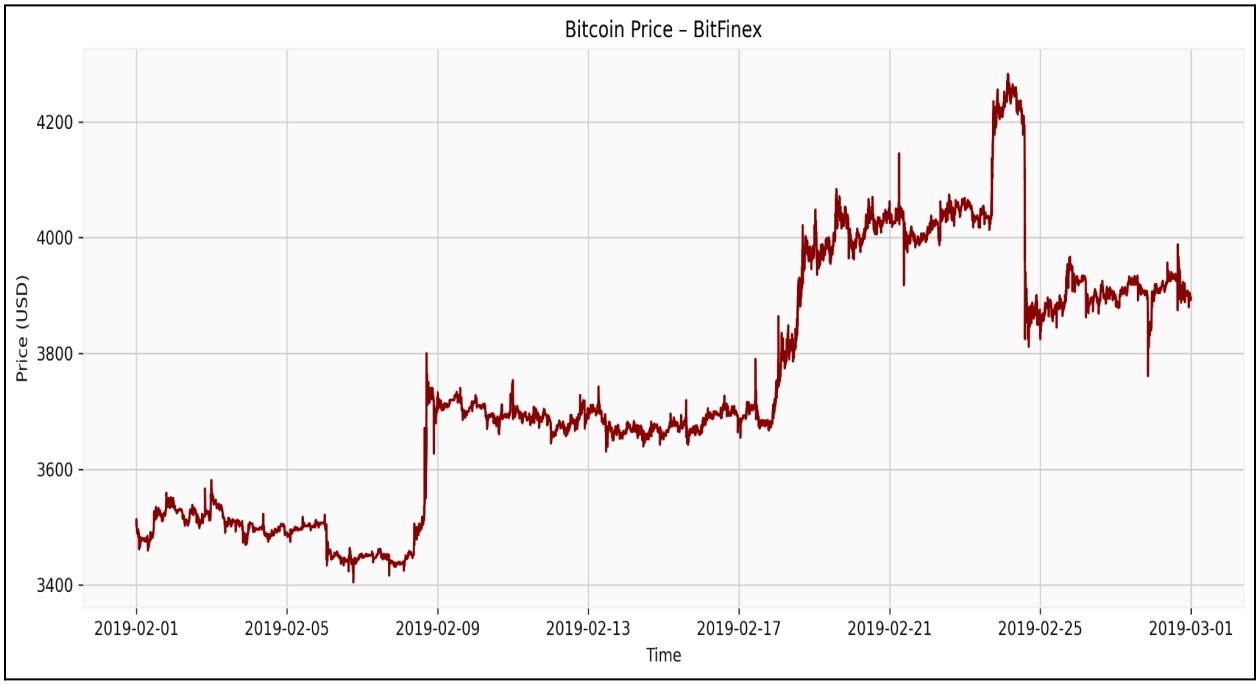

Figure 2.1 Bitcoin price time series for BitFinex

3. Anomaly Check

Visual inspections reveal occasional low-liquidity periods , particularly on smaller exchanges, where price movements appear less smooth.These periods are consistently showing reduced trading activity rather than data errors.
No extreme price spikes or different unusual patterns observed after data cleaning, indicating that the dataset is suitable for volatility analysis.

4. Daily Trading volume aggregated with exchange.

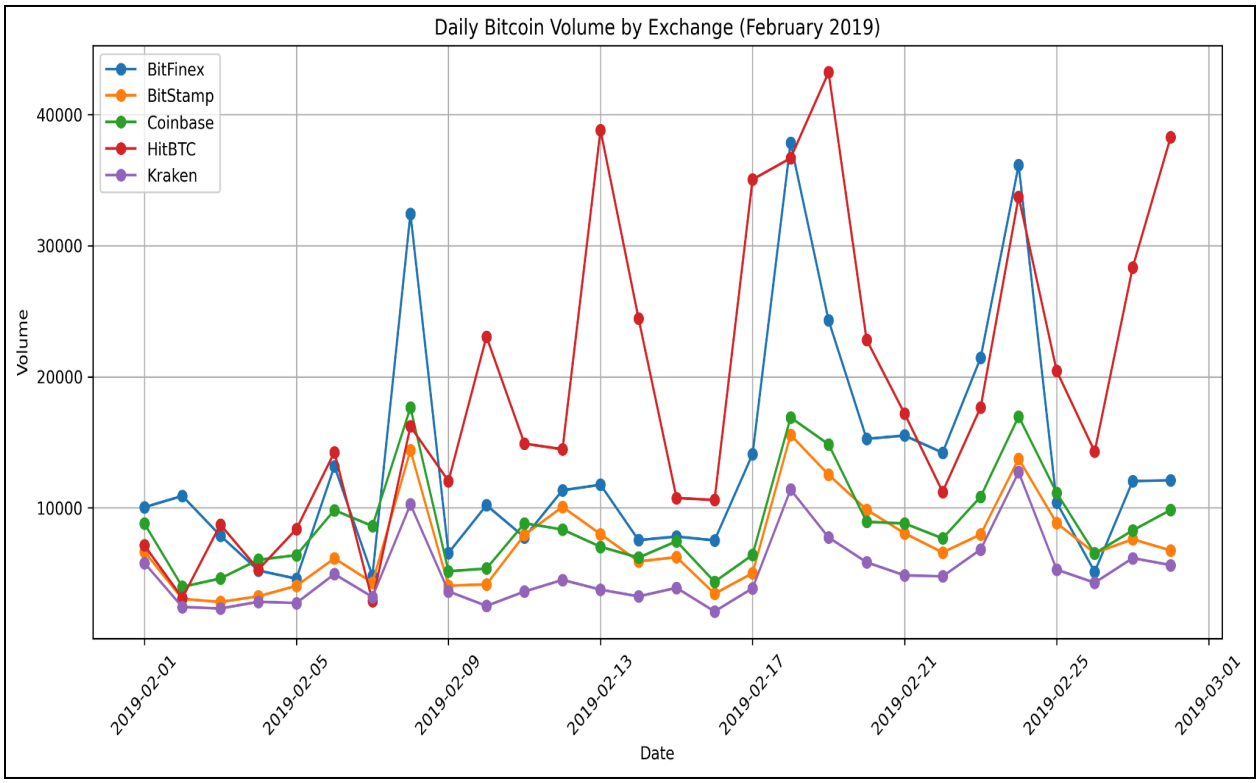

Figure 4.1 Daily Bitcoin Volume by Exchange(Line Chart)

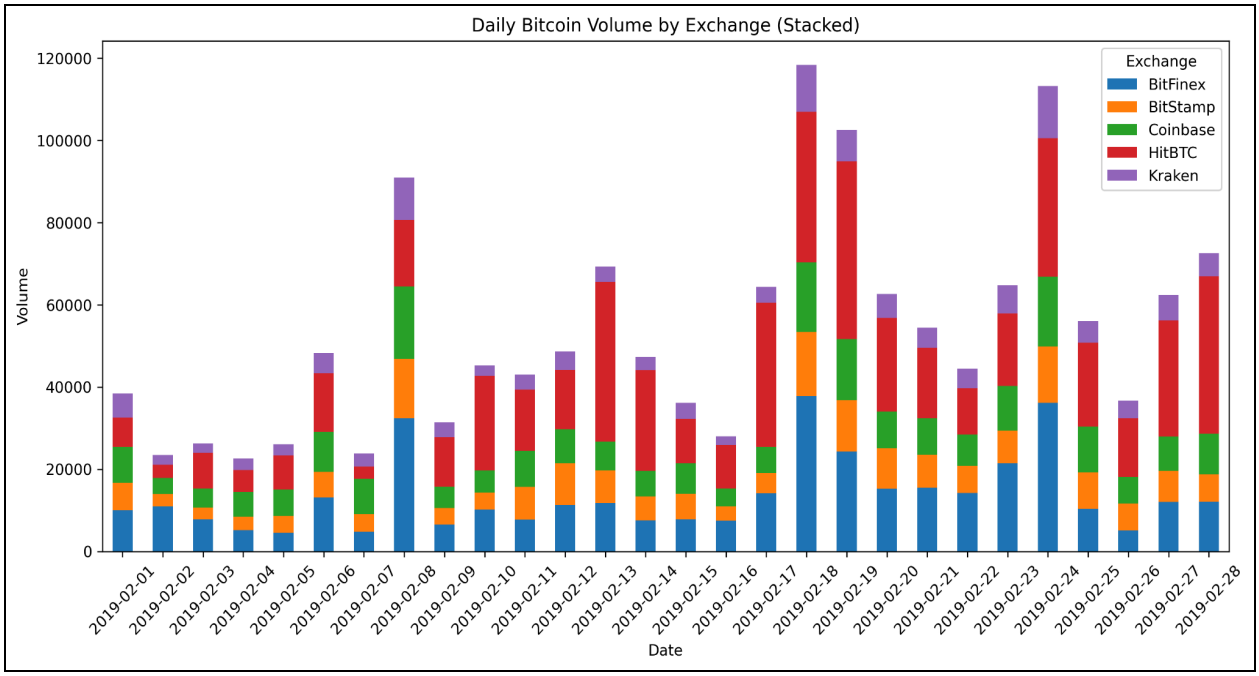

Figure 4.2 Daily Bitcoin Volume by Exchange(Stacked Chart)

The Volume Chart Indicates **HitBTC** as the highest trading volume over the sample period

5. Realized Variance Construction

5- min volume weighted average prices (VWAP) were constructed using prices and volumes with non - overlapping 5-min intervals over the 24- hour trading day. The VWAP for each interval was computed using the formula: ratio of sum of price-volume products to sum of trading volumes within that interval.

Coinbase exhibited the lowest number of missing 5-min intervals (0 missing intervals) and was therefore selected for subsequent analysis

The Highest level of daily realized volatility was observed on **24th February 2019.**
Yes- this day coincides with the reports of heightened cryptocurrency market uncertainty and increased trading activity.

Headline:
"Bitcoin suddenly dropped after reaching a multi-month high, with the price dropping from above $4,000 to around $3,800 on 24 Feb 2019."

6. Volatility Forecasting

To examine volatility predictability, several regression models were used

Using only lagged data , **Garman - Klass Volatility** shows better performance than lagged realized variance in forecasting next-period hourly realized variance.

Significance of hourly volume
Lagged hourly trading volatility is not statistically significant in predicting future realized variance

Significance of Lagged realized volatility and daily close price
Lagged daily realized volatility is statistically significant in predicting the daily closing price.

7. Visualisations

7.1

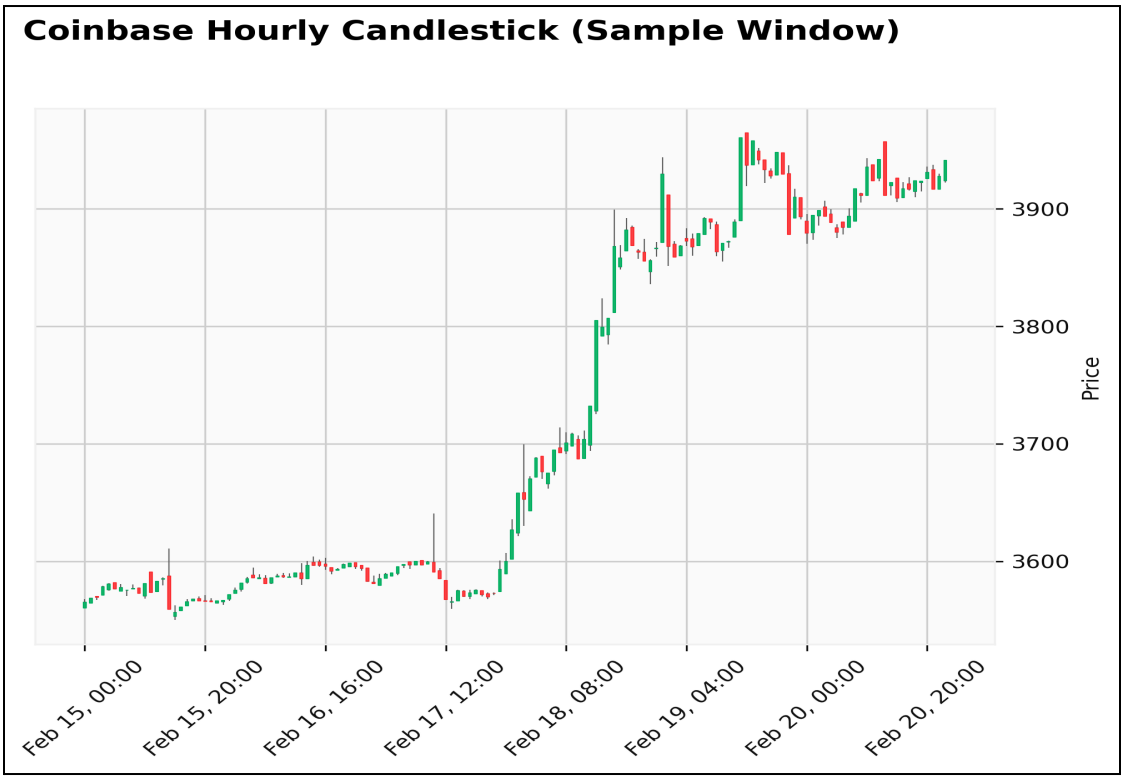

Figure 7.1 Bitcoin Coinbase price over time

Interpretation:
This figure shows the evolution of Coinbase bitcoin prices over a sample window. It indicates an overview of price trends and large market movements.

7.2

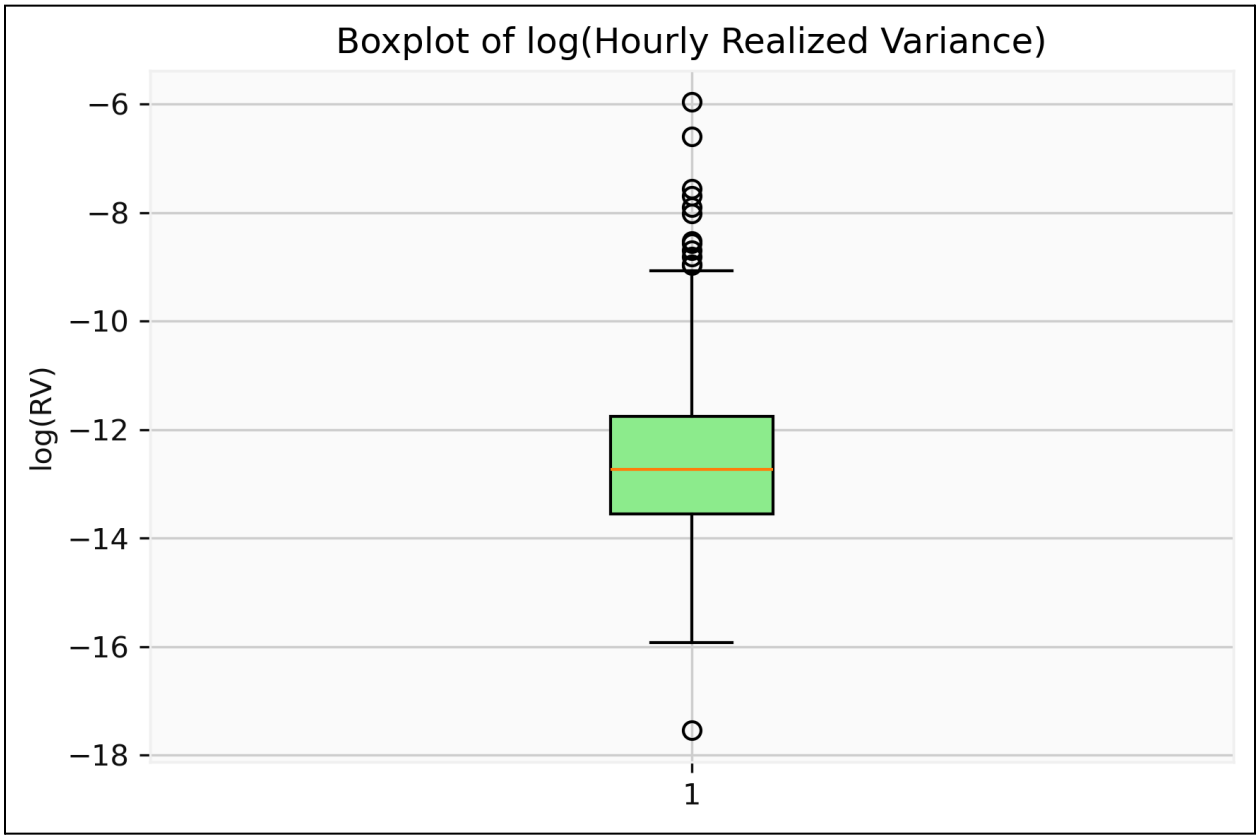

Figure 7.2 Boxplot of log hourly realized variance

Interpretation:
The chart summarizes the distribution of hourly realized variance on logarithmic scale. It displays the presence of volatility clustering and outliers.

7.3

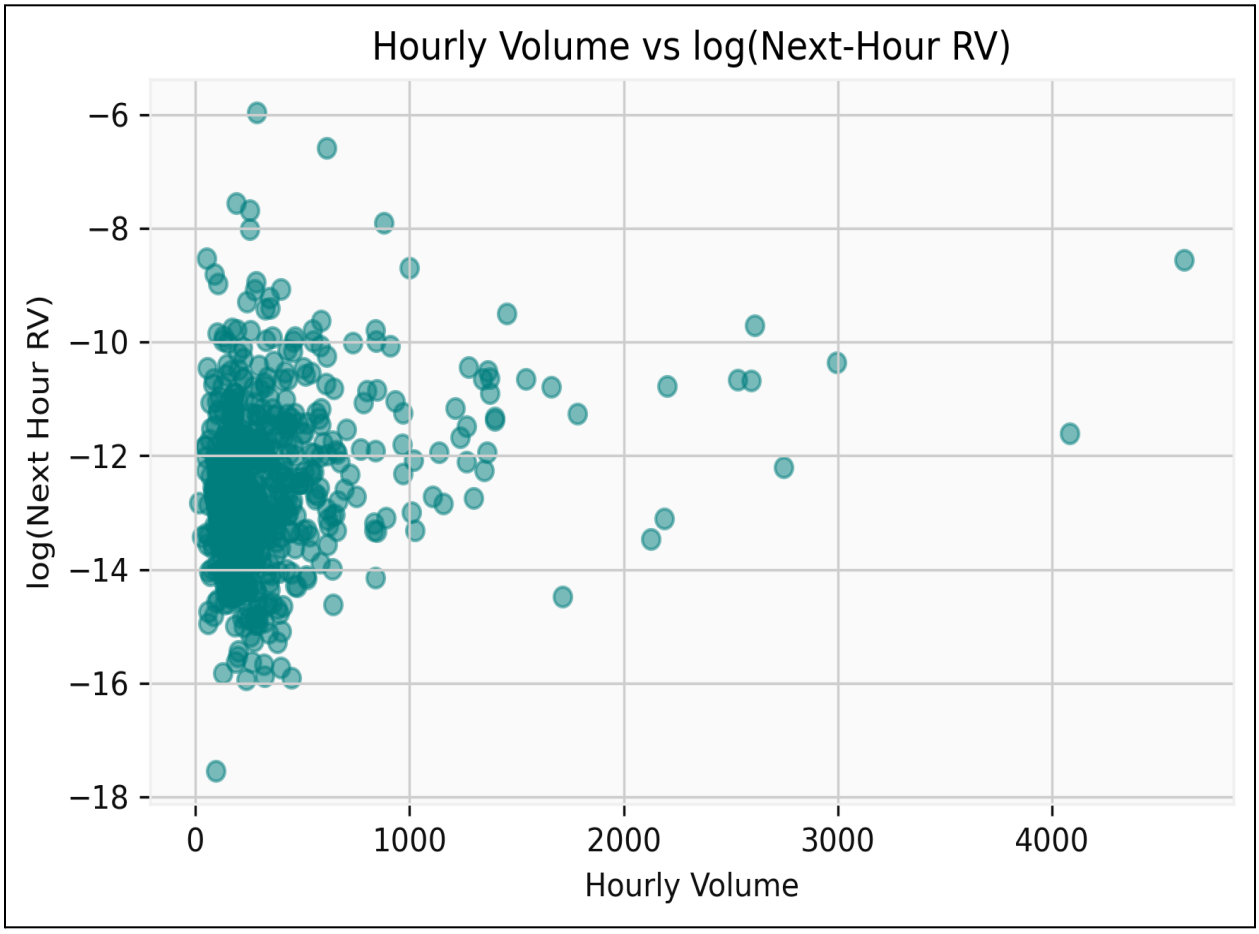

Figure 7.3 Lagged Hourly Volume vs log next-hour realized variance

Interpretation:

The scatter plot explored the relationship between trading volume and future volatility.
No strong visual relationship is seen between these 2 variables.

7.4

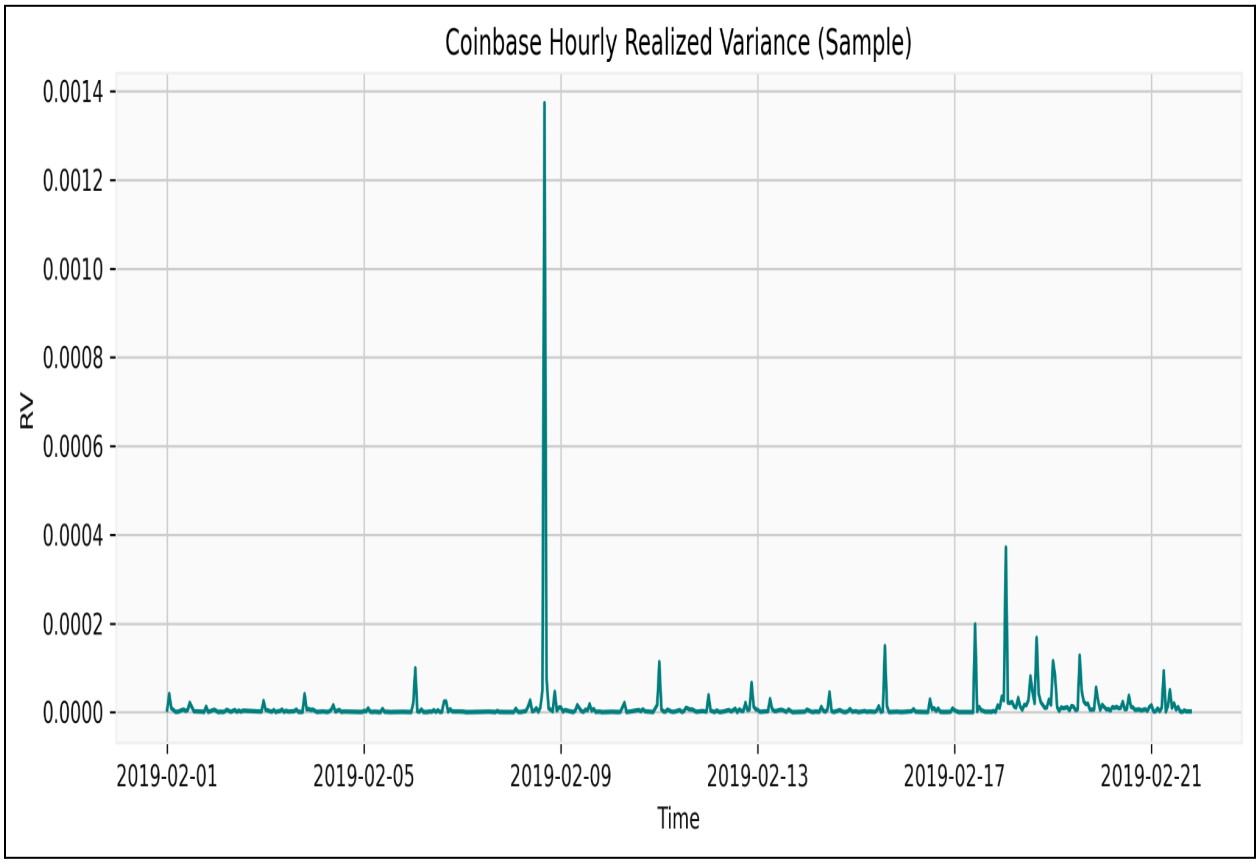

Figure 7.4 Time series of hourly realized variance

Interpretation:

This chart represents the Coinbase hourly realized variance. Periods of high volatility tend to cluster , consistent with financial market behavior.

7.5

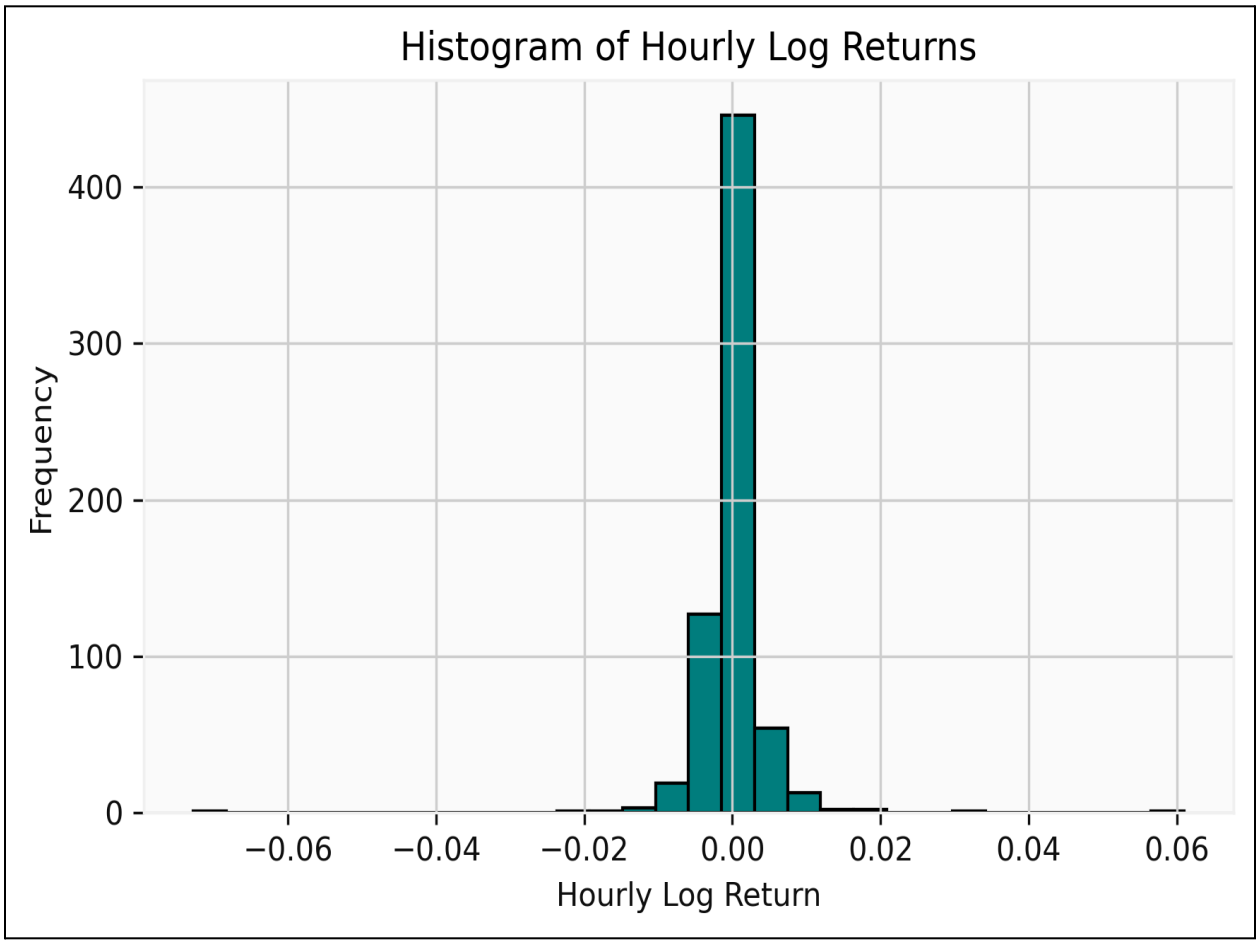

Figure 7.5 Histogram of hourly log returns

Interpretation:

Returns are centered around zero with evidence of occasional extreme values.

Reference

1. Study Material (PPTs) from Brightspace
2. Assistance of AI for coding section

8. Appendix

Part 1: Data Loading , Cleaning and Plotting

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import statsmodels.api as sm

filename= "BTC_2019-02.txt"
exchange_names = {
    1: "Coinbase",
    2: "BitFinex",
    3: "BitStamp",
    4: "HitBTC",
    5: "Kraken",
}
df= pd.read_csv(
    filename,
    names= ["timestamp","price","volume","exchange"])
df['timestamp']= (df['timestamp'].str.replace(r":(\d{3})$",r".\1", regex=True))
df['timestamp']=pd.to_datetime(df['timestamp'], format="%Y-%m-%d %H:%M:%S.%f")
df=df.set_index("timestamp")
print(df.head())

df['exchange']= df['exchange'].map(exchange_names)
df = df.dropna(subset=['exchange'])

import os
os.makedirs('figures', exist_ok=True)

color_map = {
    'Coinbase': 'darkblue',
    'BitStamp': 'darkgreen',
    'BitFinex': 'darkred',
    'Kraken': 'purple',
    'HitBTC': 'orange'}
for exch in df["exchange"].unique():
    df_exch = df[df["exchange"] == exch]
    plt.figure(figsize=(12,5))
    plt.plot(df_exch.index, df_exch["price"], color= color_map.get(exch,
'black'))
    plt.title(f"Bitcoin Price - {exch}")
    plt.xlabel("Time")
    plt.ylabel("Price (USD)")
    plt.grid(True)
    plt.tight_layout()

    plt.savefig(
        f'figures/price_{exch}.png',
        dpi=300,
        bbox_inches='tight'
    )
```

```
    plt.show()

df['date']=df.index.date
daily_volume= df.groupby(['date','exchange'])['volume'].sum().unstack()
print(daily_volume.head())

plt.figure(figsize=(12,6))

for exch in daily_volume.columns:
    plt.plot(daily_volume.index, daily_volume[exch], marker='o', label=exch)

plt.title("Daily Bitcoin Volume by Exchange (February 2019)")
plt.xlabel("Date")
plt.ylabel("Volume")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('figures/daily_volume_by_exchange_line.png', dpi=300,
bbox_inches='tight')
plt.show()

daily_volume.plot(kind='bar', stacked=True, figsize=(12,6))
plt.title("Daily Bitcoin Volume by Exchange (Stacked)")
plt.xlabel("Date")
plt.ylabel("Volume")
plt.legend(title="Exchange")
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('figures/daily_volume_by_exchange_stacked.png', dpi=300,
bbox_inches='tight')
plt.show()

total_volume = daily_volume.sum()
most_popular = total_volume.idxmax()
print(f"Most popular exchange by total volume: {most_popular}")
```

Part 2: Data Manipulations and Realized Variance estimates

```
print(df['exchange'].unique())
df.index = pd.to_datetime(df.index)
df = df.sort_index()
df.index.name = 'timestamp'

# Function to calculate VWAP
def calculate_vwap(group):
    if group['volume'].sum() == 0:
        return np.nan  # avoid division by zero
    return (group['price'] * group['volume']).sum() / group['volume'].sum()

vwap_5min_list =[]
for exch in df['exchange'].unique():
    df_exch = df[df['exchange'] == exch].copy()
    vwap = df_exch.resample('5min').apply(calculate_vwap)
```

```
    vwap = vwap.rename('vwap')# rename price column to vwap
    vwap= vwap.reset_index()
    vwap['exchange'] = exch
    vwap_5min_list.append(vwap)
vwap_5min = pd.concat(vwap_5min_list).reset_index(drop=True)
vwap_5min.head()

missing_counts = vwap_5min.groupby('exchange')['vwap'].apply(lambda x:
x.isna().sum())
print("Missing 5-min intervals per exchange:")
print(missing_counts)
best_exchange = missing_counts.idxmin()
print(f"Exchange with least missing 5-min intervals: {best_exchange}")
vwap_best = vwap_5min[vwap_5min['exchange'] == best_exchange].copy()
vwap_best.set_index('timestamp', inplace=True)
vwap_best= vwap_best.asfreq("5min")

vwap_best['vwap'] = vwap_best['vwap'].interpolate(method='time')

vwap_best['log_return']= np.log(vwap_best['vwap']).diff()
print(vwap_best.head())

#Compute squared 5-min returns
vwap_best = vwap_best.dropna(subset=['log_return'])
vwap_best['squared_return'] = vwap_best['log_return']**2
hourly_rvar = vwap_best['squared_return'].resample('h').sum()
print(hourly_rvar.head())

vwap_best['date'] = vwap_best.index.date


# Sum hourly RV per day
daily_rvar_from_hourly = hourly_rvar.resample('D').sum()

# Daily realized volatility = sqrt of daily RV
daily_rvol = np.sqrt(daily_rvar_from_hourly)

print(daily_rvol.head())

# Find the day with maximum realized volatility
max_rvol_day = daily_rvol.idxmax()
max_rvol_value = daily_rvol.max()

print(f"Day with highest realized volatility: {max_rvol_day}, RVOL =
{max_rvol_value:.6f}")

#Hourly OHLC from 5-min VWAP
ohlc_hourly = vwap_best['vwap'].resample('h').ohlc()
ohlc_hourly.head()
```

Part 3: Forecasting

```
ohlc_hourly = vwap_best['vwap'].resample('h').ohlc()
ohlc_hourly['GK_var']= 0.5*(np.log(ohlc_hourly['high']/ohlc_hourly['low']))**2\

-(2*np.log(2)-1)*(np.log(ohlc_hourly['close']/ohlc_hourly['open']))**2
```

```
# Guard against tiny negative values
ohlc_hourly['GK_var'] = ohlc_hourly['GK_var'].clip(lower=0)

# Garman-Klass volatility
ohlc_hourly['GK_vol'] = np.sqrt(ohlc_hourly['GK_var'])
print(ohlc_hourly.head())


df_best=df[df['exchange']== best_exchange].copy()
hourly_volume= df_best['volume'].resample('h').sum()
hourly_data= ohlc_hourly.copy()
hourly_data['RV_next']= hourly_rvar.shift(-1)
hourly_data['RV_lag']= hourly_rvar
hourly_data['Volume_lag'] = hourly_volume.shift(1)
hourly_data = hourly_data.dropna()
x1= sm.add_constant(hourly_data['RV_lag'])
y= hourly_data['RV_next']
model1= sm.OLS(y,x1).fit()

x2= sm.add_constant(hourly_data['GK_vol'])
model2= sm.OLS(y,x2).fit()

x3= sm.add_constant(hourly_data[['RV_lag','GK_vol','Volume_lag']])
model3 = sm.OLS(y,x3).fit()

print(model1.summary())
print(model2.summary())
print(model3.summary())

daily_close = vwap_best['vwap'].resample('D').last()

# Lagged daily realized volatility
daily_rvol_lag = daily_rvol.shift(1)  # lagged by 1 day
df_daily = pd.concat([daily_close, daily_rvol_lag], axis=1)
df_daily.columns = ['Close', 'RV_lag']
df_daily = df_daily.dropna()

#lagged daily rvol
x = sm.add_constant(df_daily['RV_lag'])
y = df_daily['Close']
model_daily = sm.OLS(y, x).fit()
print(model_daily.summary())
```

Part 4: Visualisations

```
import mplfinance as mpf

ohlc_subset = ohlc_hourly.loc['2019-02-15':'2019-02-20']

fig,ax= mpf.plot(
    ohlc_subset,
    type='candle',
    style='yahoo',
    title=f'{best_exchange} Hourly Candlestick (Sample Window)',
    volume=False,
```

```
    returnfig= True
)
fig.savefig('fig1_candlestick.png', dpi=300, bbox_inches='tight')
plt.show()

import numpy as np
import matplotlib.pyplot as plt

hourly_close = ohlc_hourly['close']
hourly_log_ret = np.log(hourly_close / hourly_close.shift(1))

# Plot histogram
plt.figure(figsize=(6,4))
plt.hist(hourly_log_ret.dropna(), bins=30, edgecolor='black', color='teal')
plt.title('Histogram of Hourly Log Returns')
plt.xlabel('Hourly Log Return')
plt.ylabel('Frequency')
plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('hist_hourly_log_returns.png', dpi=300, bbox_inches='tight')
plt.show()

#plot box plot
plt.figure(figsize=(6,4))
plt.boxplot(
    np.log(hourly_rvar.dropna()),
    vert=True,
    patch_artist=True,
    boxprops=dict(facecolor='lightgreen')
)
plt.title('Boxplot of log(Hourly Realized Variance)')
plt.ylabel('log(RV)')
plt.tight_layout()
plt.savefig('fig3_boxplot_log_hourly_rv.png', dpi=300, bbox_inches='tight')
plt.show()

plt.figure(figsize=(6,4))
plt.scatter(
    hourly_data['Volume_lag'],
    np.log(hourly_data['RV_next']),
    color='teal',
    alpha=0.5
)
plt.title('Hourly Volume vs log(Next-Hour RV)')
plt.xlabel('Hourly Volume')
plt.ylabel('log(Next Hour RV)')
plt.tight_layout()
plt.savefig('fig4_volume_vs_next_hour_rv.png', dpi=300, bbox_inches='tight')
plt.show()

sample_rv = hourly_rvar.iloc[:500]

plt.figure(figsize=(10,4))
plt.plot(sample_rv.index, sample_rv.values, color='teal')
plt.title(f'{best_exchange} Hourly Realized Variance (Sample)')
plt.xlabel('Time')
```

```
plt.ylabel('RV')
plt.tight_layout()
plt.savefig('fig5_hourly_rv_timeseries.png', dpi=300, bbox_inches='tight')
plt.show()
```